



# Блок-схема

Материал из Википедии — свободной энциклопедии

Текущая версия страницы пока не проверялась опытными участниками и может значительно отличаться от версии, проверенной 14 мая 2024 года; проверки требуют 10 правок.

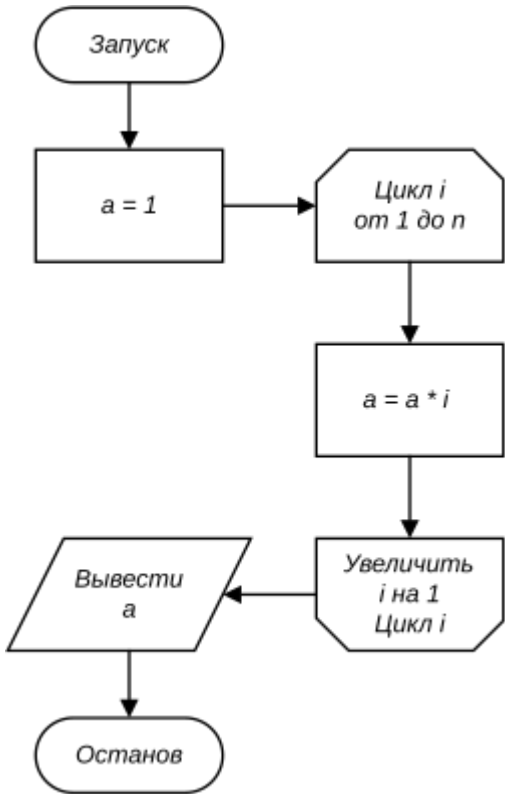
**Блок-схема** — распространённый тип схем (графических моделей), описывающих алгоритмы или процессы, в которых отдельные шаги изображаются в виде блоков различной формы, соединённых между собой линиями, указывающими направление последовательности.

## Основные элементы схем алгоритма

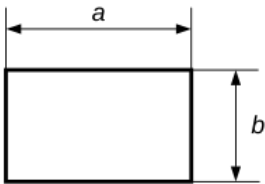
При начертании элементов рекомендуется придерживаться строгих размеров, определяемых двумя значениями *a* и *b*. Значение *a* выбирается из ряда 15, 20, 25.. мм, *b* рассчитывается из соотношения  $2 \cdot a = 3 \cdot b$ . Определение размеров несёт рекомендательный характер, однако, при соблюдении выполнения размеров блок-схемы имеют более аккуратный вид.

### Действие

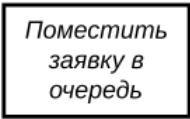
Символ отображает функцию обработки данных любого вида (выполнение определённой операции или группы операций, приводящее к изменению значения, формы или размещения информации или к определению, по которому из нескольких направлений потока следует двигаться).



Начертание

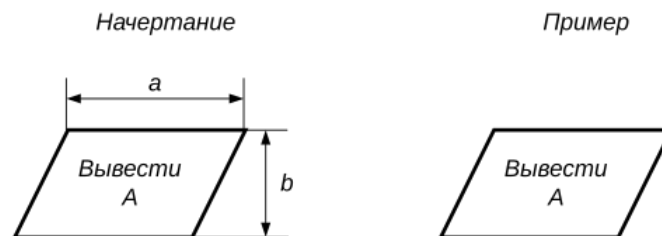


Пример



### Данные (ввод-вывод)

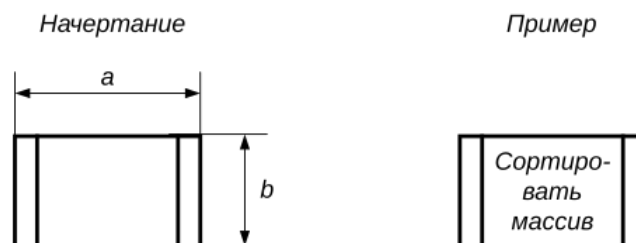
Символ отображает данные, носитель данных не определён.



Преобразование данных в форму, пригодную для обработки (ввода) или отображения результатов обработки (вывода). Данный символ не определяет носителя данных (для указания типа носителя данных используются специфические символы).

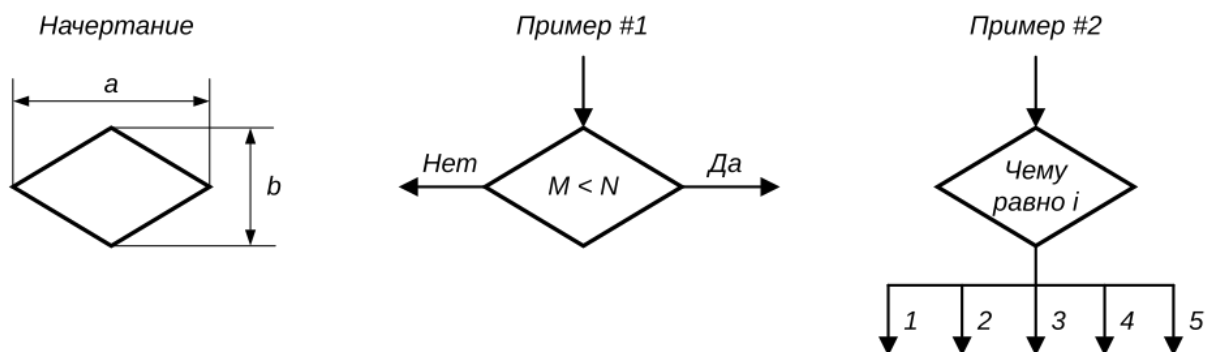
## Предопределённый процесс (функция)

Символ отображает предопределённый процесс, состоящий из одной или нескольких операций или шагов программы, которые определены в другом месте (в подпрограмме, модуле). Например, в программировании – вызов процедуры или функции.



## Вопрос (условие или решение)

Символ отображает решение или функцию переключательного типа, имеющую один вход и ряд альтернативных выходов, один и только один из которых может быть активизирован после вычисления условий, определённых внутри этого символа. Соответствующие результаты вычисления могут быть записаны по соседству с линиями, отображающими эти пути.

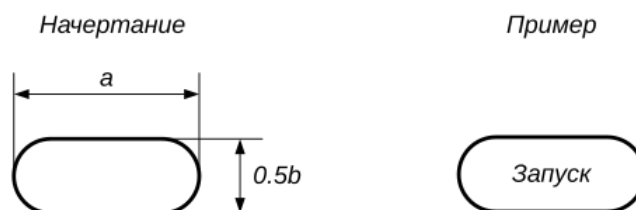


Отображает решение или функцию переключательного типа с одним входом и двумя или более альтернативными выходами, из которых только один может быть выбран после вычисления условий, определённых внутри этого элемента. Вход в элемент обозначается линией, входящей обычно в верхнюю вершину элемента. Если выходов два или три, то обычно каждый выход обозначается линией, выходящей из оставшихся вершин (боковых и нижней). Если выходов больше трёх, то их следует показывать одной линией, выходящей из вершины (чаще нижней) элемента, которая затем разветвляется. Соответствующие результаты вычислений могут

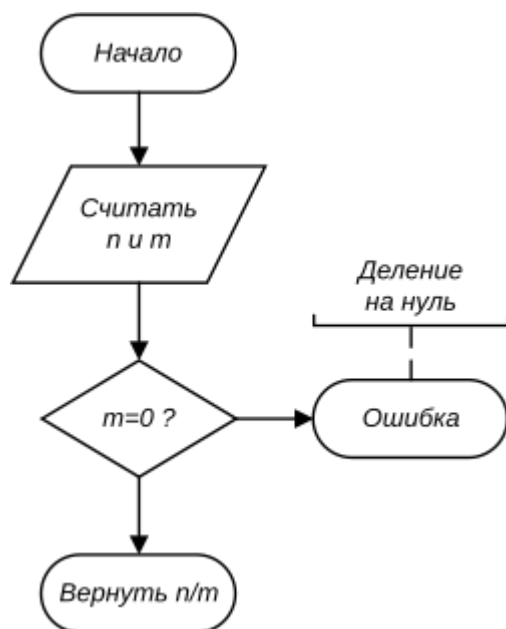
записываться рядом с линиями, отображающими эти пути. Примеры решения: в общем случае — сравнение (три выхода:  $>$ ,  $<$ ,  $=$ ); в программировании — условные операторы `if` (два выхода: `true`, `false`) и `case` (множество выходов).

## Ограничитель

Символ отображает вход из внешней среды и выход во внешнюю среду (начало или конец схемы программы, внешнее использование и источник или пункт назначения данных).

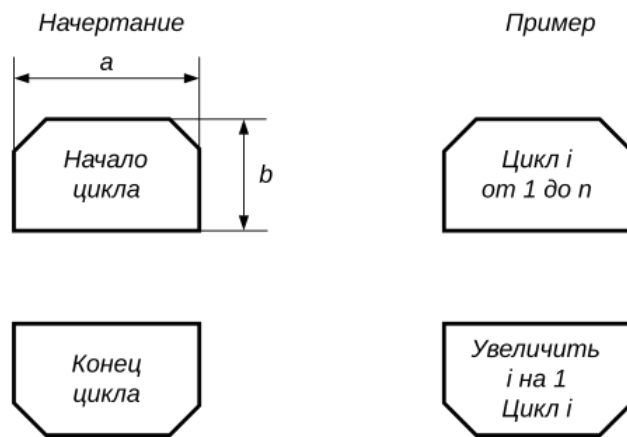


На практике имеют смысл следующие описания ограничителей: начало/конец, запуск/останов, перезапуск (подразумевает перезапуск данной блок-схемы), ошибка (подразумевает завершение алгоритма с ошибкой), исключение (подразумевает исполнение программного исключения)

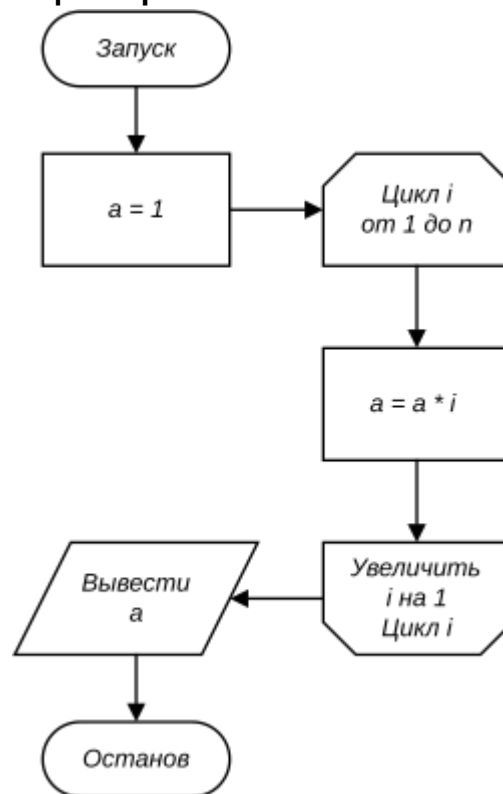


## Цикл

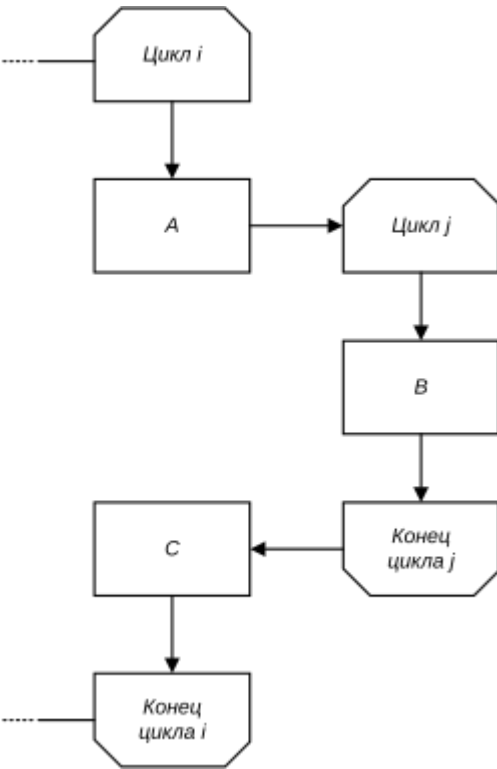
Символ, состоящий из двух частей, отображает начало и конец цикла. Обе части символа имеют один и тот же идентификатор. Условия для инициализации, приращения, завершения и т. д. помещаются внутри символа в начале или в конце в зависимости от расположения операции, проверяющей условие.



### Пример блок-схемы расчёта факториала с использованием цикла



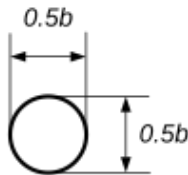
Пример вложенных циклов



Соединитель

Символ отображает выход в часть схемы и вход из другой части этой схемы и используется для обрыва линии и продолжения её в другом месте. Соответствующие символы-соединители должны содержать одно и то же уникальное обозначение.

Начертание



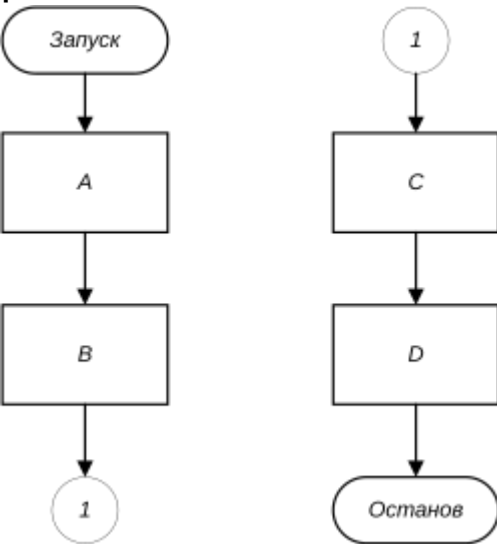
Пример #1



Пример #2



Разделение алгоритма на две части с использованием соединителей



```

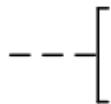
var
  s1, s2: string;
i:=1;
e:=0;
begin
  writeln('Регистрация нового пользователя');
  while i<>4 do begin
    e:=0;
    write('Введите ');
    case i of
      1: writeln('Фамилию');
      2: writeln('Имя');
      3: writeln('Отчество');
    end;
    readln(s1);
    if Length(s1)=1 then begin
      writeln('Ошибка: Длина слова меньше 2х букв');
      continue
    end;
    if (Ord(s1[1])<192) or (Ord(s1[1])>223) then begin
      writeln('Ошибка: Слово должно начинаться с русской заглавной буквы');
      continue
    end;
    for var j:=2 to Length(s1) do begin
      if (Ord(s1[j])<223) or (Ord(s1[j])>255) then begin
        writeln('Ошибка: Встречен неверный знак ',s1[j]);
        e:=1;
        break
      end;
    end;
    if e<>0 then continue;
    i:=i+1;
    s2:=s2+s1+' ';
  end;
  writeln('Здравствуйете, ',s2);
end.

```

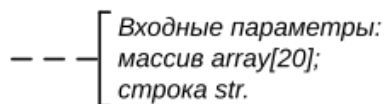
## Комментарий

Символ используют для добавления описательных комментариев или пояснительных записей в целях объяснения или примечаний. Пунктирные линии в символе комментария связаны с соответствующим символом или могут обводить группу символов. Текст комментариев или примечаний должен быть помещён около ограничивающей фигуры.

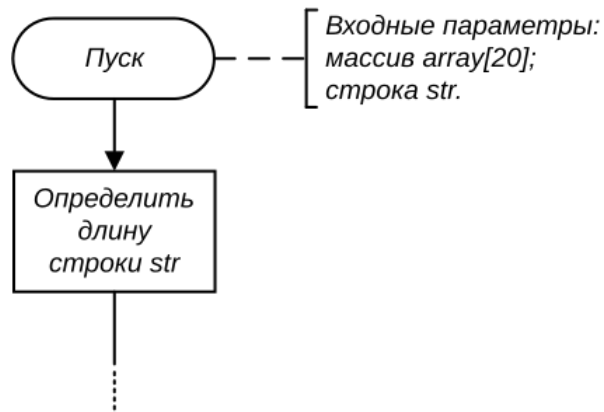
*Начертание*



*Пример*



Также символ комментария следует использовать в тех случаях, когда объём текста, помещаемого внутри некоего символа (например, символ процесса, символ данных и др.), превышает размер самого этого символа. Комментарии используют совместно с терминаторами для описания входных аргументов алгоритма при описании функций.



var s1, s2: string; i:=1; e:=0; begin

```

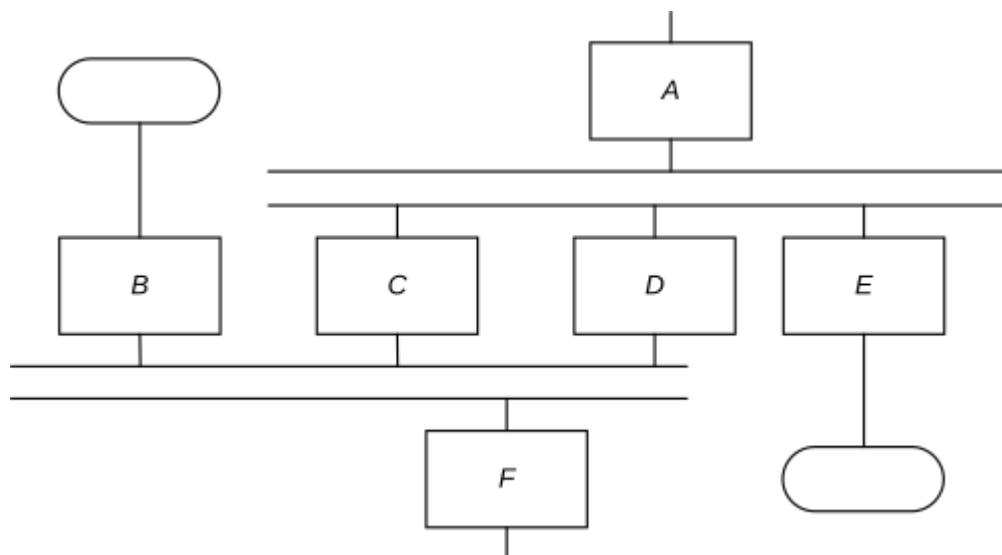
writeln('Регистрация нового пользователя');
while i<>4 do begin
  e:=0;
  write('Введите ');
  case i of
    1: writeln('Фамилию');
    2: writeln('Имя');
    3: writeln('Отчество');
  end;
  readln(s1);
  if Length(s1)=1 then begin
    writeln('Ошибка: Длина слова меньше 2х букв');
    continue
  end;
  if (Ord(s1[1])<192) or (Ord(s1[1])>223) then begin
    writeln('Ошибка: Слово должно начинаться с русской заглавной буквы');
    continue
  end;
  for var j:=2 to Length(s1) do begin
    if (Ord(s1[j])<223) or (Ord(s1[j])>255) then begin
      writeln('Ошибка: Встречен неверный знак ',s1[j]);
      e:=1;
      break
    end;
  end;
  if e<>0 then continue;
  i:=i+1;
  s2:=s2+s1+' ';
end;
writeln('Здравствуйте, ',s2);

```

end.

## Параллельные действия

Символ представляется двумя параллельными линиями, отображает синхронизацию двух или более параллельных операций. В случае входа нескольких операций в параллельные линии, выполнение алгоритма будет продолжено только в случае окончания всех входящих процессов.



Параллельные действия могут быть использованы для асинхронных процессов или для процессов, последовательность которых не важна. В представленном примере стоит обратить внимание, что созданные в одних параллельных линиях процессы не обязаны также параллельно заканчиваться.

Описание других элементов схем можно найти в ГОСТ 19.701—90<sup>[1]</sup>. Среди элементов существуют:

- Запоминаемые данные
- Документ
- Ручной ввод
- Карта
- Дисплей
- Передача управления
- Альтернативная связь между двумя или более символами
- и др.

## Представление алгоритмов в виде графов

---

Порядок выполнения действий задаётся путём соединения вершин дугами, что позволяет рассматривать блок-схемы не только как наглядную интерпретацию алгоритма, удобную для восприятия человеком, но и как ориентированный граф (т. н. граф-схема алгоритма, ГСА). Подобное представление алгоритмов используется при построении систем логического управления, реализующих заданные управляющие алгоритмы, в задачах распараллеливания вычислений и так далее.

## Критика

---

Распространённой и ошибочной практикой является попытка использования блок-схем для иллюстрации алгоритма на низком уровне (на уровне кода) — то есть попытка вписывать в блоки схемы фрагменты кода на каком-либо искусственном языке. Такой подход применим только к программам, организованным согласно структурному подходу, и не может отразить, к примеру,

алгоритм, который реализуется во взаимодействии абстракций при объектно-ориентированном подходе. Для целей описания алгоритмов, взаимодействия частей системы и иллюстрации многих других сопутствующих вещей существует нотация UML .

## Примечания

---

1. ГОСТ 19.701—90 (ИСО 5807—85) (<http://protect.gost.ru/v.aspx?control=7&id=137637>)  
«Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения»

## Ссылки

---

- <https://dgrm.net/> - online редактор блок-схем

У этой статьи **есть 2 проблемы**, помогите их **исправить**:

- В статье **не хватает ссылок на источники** (см. рекомендации по поиску). (14 октября 2011)
- Эта статья **описывает ситуацию применительно лишь к одному региону**, возможно, нарушая при этом **руководство о взвешенности изложения**. (14 октября 2011)



Пожалуйста, после исправления проблемы удалите соответствующий шаблон. Узнать, как это сделать, можно на [справочной странице](#).

---

Источник — <https://ru.wikipedia.org/w/index.php?title=Блок-схема&oldid=151992624>